

Comparação Imperativo vs POO

Resolvedor de Sudoku — Etapa 05

[P4-ETAPA-05]

Comparação entre as implementações Imperativa e Orientada a Objetos

Esta análise compara a implementação em C (imperativa, com backtracking + MRV + bitmask) e a implementação em Java (orientada a objetos, com Algorithm X e Dancing Links), tomando como base o código efetivamente produzido nas Etapas 03 e 04.

Representação do estado

- **Imperativo:** o estado central era uma variável global (`int tabuleiro[9][9]`), acessível e alterável por qualquer função do programa. O estado auxiliar da resolução (os três vetores de bitmask) vivia numa struct (`EstadoBitmask`) passada por ponteiro entre funções.
- **POO:** o estado foi distribuído entre objetos, cada um dono do seu pedaço. A grade do tabuleiro é um atributo privado de `Tabuleiro` . O estado da resolução deixou de ser um “espelho” paralelo ao tabuleiro (bitmasks) e passou a ser a própria malha de nós encadeados (`No / ColunaCabecalho`) dentro de `MatrizCoberturaExata` . Não existe mais nenhuma variável global.

Mutabilidade

- **Imperativo:** o tabuleiro era mutado diretamente a cada tentativa do backtracking (`colocarValor / removerValor` alteravam a matriz e os bitmasks em conjunto).
- **POO:** a mutação não desapareceu — ela só foi contida. `Tabuleiro.setValor` ainda muta, e as operações `cobrir() / descobrir()` do Dancing Links são, por natureza, reescrita intensa de ponteiros. A diferença é que `SudokuSolver.resolver` devolve um **novo** `Tabuleiro` como resultado, em vez de alterar o tabuleiro de entrada.

Fluxo de controle

- **Imperativo:** laços `for / while / switch` e uma única função recursiva (`resolverSudoku`) que misturava a estratégia de busca com o conhecimento das regras do Sudoku.
- **POO:** o fluxo de controle do algoritmo genérico (`buscarRecursivo` , dentro de `MatrizCoberturaExata`) está isolado do fluxo que traduz o domínio (`ConstrutorCoberturaSudoku` , inteiramente iterativo, sem recursão nenhuma). A recursão do Algorithm X não sabe o que é Sudoku.

Decomposição do problema

- **Imperativo:** decomposição por função, todas operando sobre a mesma estrutura de dados compartilhada, num único arquivo.
- **POO:** decomposição por responsabilidade e por objeto — 13 classes/interfaces distribuídas em três pacotes (`Objeto` , `validar` , `resolver`), cada um cuidando de uma fatia distinta do problema.

Reutilização

- **Imperativo:** havia reaproveitamento pontual (`lerTabuleiro` recebendo o destino por parâmetro), mas `resolverSudoku` estava inteiramente acoplado ao Sudoku — não reaproveitável para nenhum outro problema.
- **POO:** `MatrizCoberturaExata` é genuinamente genérica (não menciona Sudoku em nenhuma linha) e poderia resolver qualquer problema de cobertura exata com um novo construtor. `Validador` reaproveita qualquer `RegraValidacao` plugada nele.

Manutenção

- **Imperativo:** alterar uma regra do jogo exigia editar funções monolíticas (`ValTabuleiro` , `resolverSudoku`) que já faziam várias coisas ao mesmo tempo.
- **POO:** alterar uma regra de validação isolada é tocar só a classe daquela regra (`RegraLinha` , `RegraColuna` ou `RegraBloco`). Um ponto de dívida identificado: as regras do Sudoku estão descritas em dois lugares independentes (as `RegraValidacao` , para validar, e `ConstrutorCoberturaSudoku` , para montar a matriz de cobertura) — uma mudança de regra precisaria ser replicada nos dois.

Facilidade de extensão

- **Imperativo:** baixa — qualquer regra nova exigiria reescrever a função de validação ou de resolução inteira.
- **POO:** alta para validação (uma nova regra é só uma nova classe implementando `RegraValidacao` , sem tocar em nada existente); média para resolução (trocar a estratégia de busca exigiria primeiro extrair uma interface de `SudokuSolver` , o que não foi feito).

Tratamento de erros

- **Imperativo:** códigos de retorno (`0 / 1`) e mensagens impressas diretamente dentro da função que detectou o problema (ex: `lerTabuleiro` já imprimia o erro antes de retornar).
- **POO:** uso de exceções (`IOException`). A função que lê o arquivo apenas lança a exceção com uma mensagem; quem decide como exibir isso ao usuário é o chamador (`Aplicacao`), separando “detectar o erro” de “reportar o erro”.

Efeitos colaterais

- **Imperativo:** espalhados — qualquer função recebendo a matriz por parâmetro podia alterá-la.
- **POO:** confinados — mutação só acontece dentro de métodos pertencentes ao objeto dono daquele estado (`Tabuleiro.setValor` , ou `No / ColunaCabeçalho` dentro do pacote `resolver`).

Facilidade para testar

- **Imperativo:** difícil isolar uma função para teste unitário, já que várias dependiam do estado global ou de listas encadeadas montadas e destruídas manualmente (`malloc / free`) dentro da própria função.
- **POO:** mais fácil isolar — `RegraLinha` , `RegraColuna` e `RegraBloco` podem ser testadas independentemente umas das outras, assim como `Tabuleiro` , cada uma com um contrato pequeno e claro.

Organização do código

- **Imperativo:** um único arquivo (`ImpParadigmas.c`) concentrando todas as structs e funções.
- **POO:** múltiplos arquivos organizados em pacotes (`Objeto` , `validar` , `resolver` , `Principal`), cada um refletindo uma fronteira de responsabilidade.

Complexidade

Usando `N` como o tamanho do tabuleiro (`N = 9` no caso clássico, com blocos $\sqrt{N} \times \sqrt{N}$), para poder expressar a complexidade de forma genérica em vez de só “é rápido o suficiente para 9x9”:

Validação

- *Imperativo (`ValTabuleiro`):* uma única passada. Para cada um dos `N` dígitos, percorre as `N2` células (`O(N3)` no total) montando a lista de posições, e depois compara os pares encontrados (um número pequeno e limitado por dígito, então não altera a ordem de grandeza). Complexidade total: **`O(N3)`**.
- *POO (`RegraLinha` + `RegraColuna` + `RegraBloco`):* cada regra, isoladamente, já custa `O(N3)` (dígito $\times$ linha $\times$ coluna), e as três rodam de forma independente. Complexidade total: **`O(3·N3)`** — mesma ordem de grandeza assintótica do C (`O(N3)`), mas com um fator constante 3x maior, porque o tabuleiro é varrido três vezes (uma por regra) em vez de uma vez só. A deduplicação final com `LinkedHashSet` custa `O(1)` amortizado por inserção, então não muda essa análise.

Resolução

Sudoku generalizado para um tabuleiro `N×N` é um problema **NP-completo** (não existe algoritmo de tempo polinomial conhecido para resolvê-lo em geral). Isso significa que, no pior caso teórico, **tanto o backtracking quanto o Algorithm X são exponenciais** — nenhum dos dois paradigmas muda a classe de complexidade do problema:

- *Imperativo (`backtracking` + `MRV` + `bitmask`):* pior caso **`O(Nm)`**, onde `m` é o número de células vazias (até `N2`) e `N` é o número de dígitos candidatos por tentativa. A heurística `MRV` e o `bitmask` não mudam esse limite teórico — eles reduzem o fator de ramificação *na prática* (podam ramos cedo) e tornam cada consulta de candidatos **`O(1)`** (operações `|` , `~` , & sobre inteiros), em vez de `O(N)` (percorrer linha/coluna/bloco manualmente a cada tentativa).
- *POO (`Algorithm X com Dancing Links`):* mesmo pior caso teórico, **`O(Nm)`** — é o mesmo problema NP-completo, resolvido por um algoritmo de busca exaustiva com poda, assim como o `backtracking`. A diferença está no **custo por operação dentro da busca**: cobrir/descobrir uma coluna custa `O(k)` , onde `k` é o número de nós afetados (no máximo `N` jogadas candidatas $\times$ 4 colunas por jogada, uma constante pequena para `N` fixo) — ou seja, também efetivamente **`O(1)`** por chamada, para tabuleiros de tamanho fixo. A escolha da coluna mais restrita (`escolherColuna`) é estruturalmente a mesma heurística `MRV` do C, só aplicada sobre “restrições” em vez de “células”.

Conclusão sobre complexidade: a troca de paradigma **não mudou a ordem de grandeza** de nenhuma das duas partes do problema — validação continua `O(N3)` nos dois casos, e resolução continua exponencial (`O(Nm)`) nos dois casos, por ser inerente ao problema, não à implementação. O que mudou foi (a) um fator constante maior na validação em Java (3 passadas em vez de 1) e (b) a estrutura de dados usada para conseguir custo `O(1)` /operação na resolução — `bitmask` (vetores de inteiros com operações bit a bit) no C, versus uma malha de nós encadeados (`Dancing Links`) no Java. A orientação a objetos não tornou o algoritmo mais rápido nem mais lento em essência; tornou a complexidade **estrutural** (número de classes, indireção entre objetos) maior, em troca de melhor organização e separação de responsabilidades.

Perguntas

1. Qual problema ficou mais fácil de expressar de forma imperativa?

Toda a parte de implementar o algoritmo era bem mais fácil no paradigma imperativo que o orientado a objetos.

2. Qual problema ficou mais fácil de expressar utilizando orientação a objetos?

A orientação a objetos se tornou mais fácil em pequenos pontos, como tornar menos verbosa a alocação de memória para listas e preparar os objetos para impressão.

3. Onde a orientação a objetos realmente trouxe vantagem?

A orientação a objetos trouxe vantagem especialmente na organização do código como um todo, permitindo separar por arquivos cada classe necessária para a resolução do problema.

4. Em quais situações a utilização de objetos acrescentou complexidade desnecessária?

A utilização de objetos se tornou desnecessária principalmente no algoritmo de validação do tabuleiro, uma vez que foi necessário adicionar o uso de tabelas hash para diferenciar erros repetidos.

5. Que partes do problema praticamente não mudaram entre as duas implementações?

A parte do método principal (`main`), uma vez que ele era basicamente aquele que definia quando cada função seria chamada para cada ação pedida.

6. Que partes precisaram ser completamente remodeladas?

A parte do algoritmo de resolução do Sudoku, uma vez que busquei uma abordagem mais genérica do que a utilizada no paradigma imperativo.